

Evaluating the Quality of GenAI Applications in Software Engineering: A Multi-case Study

Liang Yu · Emil Alégroth · Panagiota Chatzipetrou · Tony Gorschek

Received: date / Accepted: date

Abstract *Context:* Generative AI (GenAI) is increasingly adopted in software development for tasks such as document generation, data analysis, and code generation. However, evaluating the quality of GenAI applications becomes challenging, as traditional quality measurements may not be fully applicable.

Objective: In this study, we explore how practitioners evaluate the quality of GenAI applications and investigate quality evaluation techniques.

Method: We conducted a multi-case study in three industrial projects from software development companies. We examined four GenAI application domains: document generation, data analysis and insight generation, customer service, and code generation. Data were collected through three workshops and 23 semi-structured interviews with industrial practitioners.

Results: We identified fourteen GenAI use cases and 28 metrics currently used to evaluate the quality of GenAI applications' outputs. We synthesized the identified metrics' usage patterns and challenges based on the collected data.

Conclusions: This study presents practical insights into using metrics to measure GenAI-based system qualities in real industrial settings. Our findings indicate that practitioners use custom-built and context-specific metrics;

Liang Yu
Blekinge Institute of Technology, Valhallavägen 10, 371 79 Karlskrona, Sweden
E-mail: liang.yu@bth.se

Emil Alégroth
Blekinge Institute of Technology, Valhallavägen 10, 371 79 Karlskrona, Sweden
E-mail: emil.alegroth@bth.se

Panagiota Chatzipetrou
Örebro University, Fakultetsgatan 1, 702 81 Örebro, Sweden
E-mail: panagiota.chatzipetrou@oru.se

Tony Gorschek
Fortiss, Guerickestr 25, 80805 Munich, Germany
Blekinge Institute of Technology, Valhallavägen 10, 371 79 Karlskrona, Sweden
E-mail: tony.gorschek@bth.se

combining these with academic metrics can strengthen GenAI system quality evaluation.

Keywords Generative Artificial Intelligence · GenAI · Large Language Model · LLM · Metric · Quality Evaluation

1 Introduction

The rapid adoption of Generative Artificial Intelligence (GenAI) in the industry has revolutionized tasks such as chatbot, information retrieval, code review/generation, and anomaly detection [4,3]. The industrial interest in utilizing GenAI reflects a desire to streamline and improve the processes of tasks traditionally requiring significant human effort [7]. For instance, GenAI-powered *chatbot* services have been widely integrated into industrial web interfaces to handle user queries [6]. Similarly, GenAI models support *information retrieval* from large documents [25], assist in *code review* by highlighting issues such as security vulnerabilities [3,42], and identify *financial anomalies* in source files within minutes [4].

However, evaluating the quality of these GenAI applications remains a challenge. In this paper, we use the term *GenAI applications* to refer to systems that support software development activities by integrating generative AI models for tasks such as document generation, trouble report analysis, and code generation. Our focus lies in understanding how GenAI technologies are adopted in ongoing software development projects and how the quality of these applications is evaluated in industrial practices. Traditional software quality assurance approaches are mostly designed for deterministic systems with transparent logic and traceable outputs. In contrast, GenAI systems produce probabilistic, non-deterministic responses [29]. They often function as ‘black boxes’, making conventional evaluation techniques inadequate [30]. This shift in quality assurance calls for new or adapted evaluation approaches that better reflect the unique characteristics of GenAI outputs [24].

In response, prior research has proposed a variety of experiment-driven metrics to assess GenAI systems. These metrics, including accuracy, relevance, and consistency, aim to quantify output quality based on reference-based or task-based scoring [5]. In this study, we refer to such metrics as *experiment-driven metrics*, developed within academic research contexts for controlled evaluation purposes. However, it remains unclear whether these metrics can be effectively applied in industrial environments, given constraints related to scale, domain specificity, and the lack of labeled ground-truth data. GenAI systems deployed in practice often require adaptation to domain-specific language or tasks [5]. For example, a pre-trained model may perform poorly on legal or financial documents, prompting organizations to adopt custom evaluation criteria tailored to their operational goals. Consequently, evaluating GenAI system outputs often depends on use-case-specific quality requirements that standard research metrics may not adequately capture.

To investigate industrial practices, we studied four GenAI application domains in software development companies: document generation, data analysis and insight generation, customer service chatbots, and code generation. These domains were identified through engagement with industrial engineers and are also highlighted in recent literature on GenAI adoption in industry, such as Esposito et al. [10] and Kenthapadi et al. [18]. Feldt et al.’s AI-SEAL taxonomy [12] further categorizes how GenAI is integrated into software development, including human-in-the-loop, automated, and AI-assisted workflows, which align with the evaluation patterns observed in our study.

To understand how quality is evaluated in real-world GenAI applications, we conducted a multi-case study across three software development companies. We gathered insights through workshops and semi-structured interviews with engineers directly involved in GenAI implementation and evaluation. Our goal was to identify the quality metrics used, the methods of applying them, and the challenges encountered during evaluation.

The main contributions of this study are:

- An overview of metrics currently used in the industry for GenAI quality measurement.
- Insights into the practical use of these metrics within specific industrial contexts and suggestions for utilizing experiment-driven metrics.
- Challenges when using metrics to evaluate the quality of GenAI applications.

Our findings reveal that quality evaluation in industry is highly context-specific. Practitioners rely on lightweight, task-oriented checks that reflect their domain needs and project constraints. While experiment-driven metrics from research are rarely used directly, several are seen as promising when appropriately adapted. We also identify distinct evaluation challenges, which we translate into actionable recommendations for practitioners seeking to assess GenAI systems’ quality.

The rest of this paper is structured as follows: Section 2 introduces related work. Section 3 illustrates the research methodology. Section 4 presents the study results based on extracted data. Section 6 discusses threats to the study’s validity. Section 7 discusses the findings of our research. The conclusions and future work are in Section 8.

2 Related work

2.1 GenAI in Software Engineering

Generative AI (GenAI) technology has been used for requirement analysis, documentation, bug triaging, code generation, and testing [14] in Software Engineering (SE). Bommasani et al. [3] and Zhang et al. [42] reviewed these areas and noted GenAI’s automation potential across several development stages. Kenthapadi et al. [18] reported practical challenges observed in industrial adoption, such as domain alignment and model hallucination. Esposito

et al. [10] explored the use of GenAI to support various phases of software engineering, including requirements-to-architecture and architecture-to-code transformations. These findings support the relevance of the domains analyzed in our study. Feldt et al. [12] introduced the AI-SEAL taxonomy, which classifies AI applications in software engineering based on integration depth and human involvement. Their taxonomy reinforces our evaluation framework and highlights current industrial patterns in GenAI adoption.

While these studies demonstrate the growing relevance of GenAI in SE, few address how GenAI system qualities are evaluated in practice, especially under the non-deterministic and probabilistic nature of GenAI outputs.

2.2 Evaluation metrics for GenAI in Software Engineering

In software engineering, ISO/IEC 25010 and ISO 25023 [16] define quality characteristics, such as functional suitability, reliability, and maintainability. Research work (e.g., [26] [21] [33] [23]) has employed metrics [19] like accuracy, relevance, and consistency to score GenAI-generated outputs [5]. Most of these metrics are tested in controlled experiment settings and lack validation across industrial domains. The remainder of this section reviews metric usage in four application domains.

2.2.1 Document generation domain

Document generation tasks, such as summarizing requirements, drafting design documents, and generating implementation proposals, are common areas of GenAI application in software engineering [3, 10]. In academic settings, such tasks are often evaluated using metrics adapted from natural language generation, including BLEU [26], ROUGE [21], and METEOR [2]. These metrics quantify text-level similarity between generated and reference texts, but may not adequately capture semantic correctness or task-specific value. Recent metrics, such as BERTScore [41] and FActScore [23], aim to assess semantic alignment and factual consistency. These metrics rely on source data citations, which require intensive time and effort to be manually prepared in industrial settings. Despite these academic metrics, little is known about their actual usage in GenAI-enabled software projects.

2.2.2 Data analysis and insight generation domain

GenAI can be helpful in log interpretation, incident correlation, trend identification, and insight extraction from historical defect reports [42, 18]. However, evaluating the relevance of such insights is particularly difficult in industrial contexts, because a relevant output depends on domain-specific expectations and often lacks predefined reference data for comparison. Academic methods often use F1-score [19] to evaluate the relevance. When GenAI ranks outputs (e.g., most likely causes or top anomalies), *Precision* is often used [6].

Some approaches frame the task as supervised classification, using labeled categories [14]. These metrics rely on structured validation datasets; therefore, manual review, user feedback, or correctness checks are needed in these scenarios.

2.2.3 Customer service chatbots domain

GenAI chatbots can handle support queries and ticket triage in IT service domains [18, 25]. Quality assessment in this domain contains *response quality* and *user experience* dimensions. Technical criteria include correctness and relevance. Evaluation methods in research use both automatic and human-involved approaches. Automatic metrics include BLEU [26] and ROUGE [21] for response generation, as well as more chatbot-specific measures such as F1-score for *dialogue success rate* [42]. Recent works evaluate *factual consistency* using FActScore [23] or similar metrics. Human-centered evaluations often use rating scales to assess helpfulness [5]. In industrial settings, evaluations involve user satisfaction scores and support logs.

2.2.4 Code generation domain

GenAI coding assistants support code completion, refactoring, documentation, and automated test generation [42, 3]. Evaluating generated code involves assessing both functionality and maintainability. Research employs PASS@k [15] (execution success of generated code), CodeBLEU [33], and *Exact Match* [6] to assess the quality of GenAI-generated code. Static analysis metrics such as *cyclomatic complexity*, *code duplication*, and *lint violations* can also be employed, especially when code is not executable. However, these metrics may miss system-level concerns like explainability, transparency, or trust.

3 Research methodology

We conducted a multi-case study inspired by Runeson et al.’s guidelines [34]. The case study methodology was chosen because it allows for an in-depth exploration of use cases and deeper insights into the quality measurement practices, metrics used, and challenges practitioners face for GenAI-supported software development. An overview of research processes is shown in Fig. 1.

3.1 Step 1 - Plan

In this phase, we formulated research objectives and research questions.

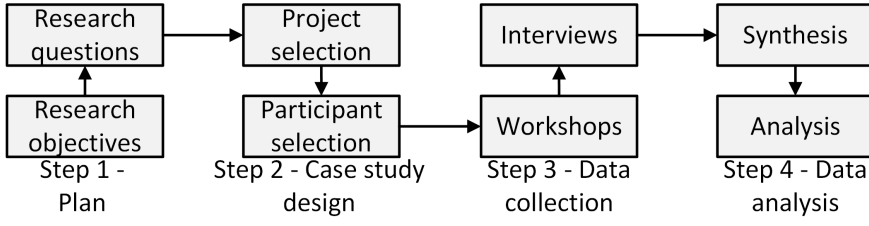


Fig. 1 Overview of research processes.

3.1.1 Research objectives

This study aims to I) understand how GenAI is currently being applied in industrial software development settings; II) explore how practitioners evaluate the quality of outputs produced by GenAI-enabled systems; III) identify challenges and contextual factors that influence the use of metrics in these environments.

3.1.2 Research questions

To achieve the research objectives, we formulated the following research questions:

- RQ1: What metrics are used to evaluate the quality of GenAI applications?
- RQ2: What types of evaluation methods are used to measure the identified metrics for GenAI applications?
- RQ3: What challenges are encountered in applying metrics to evaluate the quality of GenAI applications?

3.2 Step 2 - Case study design

This section presents the contexts of the industrial projects and the study participants' selection.

3.2.1 Project selection

Industrial cases were sampled using purposive sampling [27] from our existing industrial network, specifically companies that were, at the time of the study, engaged in developing GenAI applications. This purposive sampling ensured that the selected organizations – serving as the units of analysis – had ongoing experience with GenAI in software development. Our focus was on organizations that had already integrated GenAI capabilities into real-world projects, so that we could investigate how system qualities were evaluated in practice, rather than in experimental or theoretical contexts [27]. Furthermore,

the study is descriptive, aiming to describe the use cases that the purposively sampled organizations currently have for GenAI.

Project A operates in the FinTech domain, focusing on mobile digital payments, while Projects B and C are in the telecommunication domain, providing operational support services and network management.

Table 1 presents the context information of the selected industrial projects. According to the guidelines set by Petersen et al. [28], each context is described by its business domain, product type, maturity, and organizations.

Table 1 Context information [28] of the selected industrial projects

Context aspect	Context element	Project A	Project B	Project C
<i>Project</i>	Business domain	FinTech	Telecom billing	Telecom network
	Product type	Finance services	Operation support services	Network management
	Maturity of product	Mature product	Mature product	Mature product
<i>Organization</i>	number of engineers	>500	>1000	>200
	Distributed development	Yes (global)	Yes (global)	No (nationally)
<i>GenAI</i>	Usage of GenAI	Yes	Yes	Yes
<i>Quality attributes</i>	ISO 25023	Security Usability Reliability	Functional suitability Usability Reliability Maintainability	Usability Reliability

As can be seen in Table 1, there are several differences in business domains, team structures, and quality focus areas across the selected projects. This selection thereby offers a foundation for exploring varied approaches to GenAI quality evaluation. Additionally, this diversity enhances the generalizability of our findings, providing an understanding of the applicability of different metrics in various industrial contexts. However, we stress that the results are still captured from a limited sample, albeit diverse, which may limit our findings, as further discussed in Section 6.

3.2.2 Participant selection

We used purposive sampling [38] to recruit participants through industrial contacts at the case companies. The selected participants were actively involved in GenAI-related development, integration, or evaluation tasks. The same group of participants was involved in both the workshops and follow-up interviews, enabling continuity across data collection phases and allowing us to deepen and validate the findings through multiple engagement formats.

As shown in Table 2, 30 participants were chosen. Participant roles included software developers, managers, software architects, security engineers, opera-

tions engineers, and document engineers. Their working experience ranges from 4 to 25 years.

Table 2 Participants from the selected industrial projects A, B, and C

ID	Participant role	NO. of participants	Working experience (years)	Project
1	Software developer	6	5 - 20	A (2), B (2), C (2)
2	Manager	5	5 - 25	A (1), B (2), C (2)
3	Software architect	5	7 - 23	A (2), B (2), C (1)
4	Security engineer	5	5 - 17	A (1), B (3), C (1)
5	Operation engineer	5	4 - 15	A (2), B (1), C (2)
6	Document engineer	4	5 - 10	A (1), B (1), C (2)

For the workshops, the focus was on a broad understanding of how GenAI can be specified, integrated, and tested in industrial software applications. Participants provided input on the use cases and their contexts during these collaborative sessions. The workshops were complemented with interviews that aimed to gather more detailed insights into the metrics used for quality evaluation in the identified use cases.

To mitigate threats to external validity [20], we applied purposive sampling to select three software companies with variation in industry domain, company size, and maturity of GenAI adoption. This approach allowed us to explore a range of GenAI application scenarios while ensuring relevance to practical settings. Construct validity was also considered during this phase by defining key concepts (e.g., “GenAI use case,” “quality evaluation”) internally among the research team and validating them through a pilot interaction with one participating company.

3.3 Step 3 - Data collection

This section presents the workshop and interview processes of the data collection.

3.3.1 Workshop design

All authors participated in designing the workshop, which involved three distinct phases developed through four rounds of meetings and refinements (see Fig. 2). In total, three workshops were executed separately in Projects A, B, and C to address the specific contexts of each project.

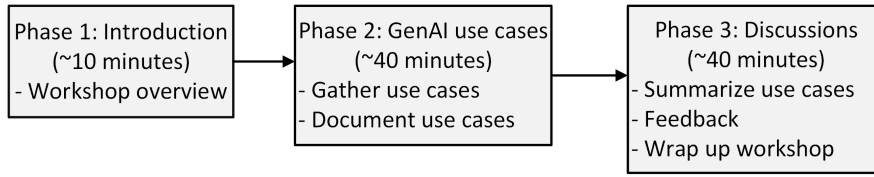


Fig. 2 Overview of the workshop process

Phase 1: Introduction (approx. 10 minutes). The workshop began with an introduction, during which we explained the purpose and objectives of the study. Explicitly, the participants were told that the study focused on gathering experiences from use cases where GenAI was applied, explored, or tested. Hence, speculative or future use cases, which had not yet been applied in the projects, were not in scope for this study. This phase ensured participants understood the workshop’s goals and their role in contributing relevant insights.

Table 3 A template for documenting GenAI use cases [1]. An example inspires participants to fill in their use case details.

Template for a LLM use case	
Use case name	E.g., Software upgrade path assistant
Use case description	E.g., A LLM chatbot assists business analysts to generate report about product feature/interface changes between releases.
Target user(s)	E.g., Business Analyst (BA), Solution Architect (SA)
Challenge(s)/Issue(s) - resolved by this use case	E.g., Business analysts/Solution analysts often struggle to maintain a clear understanding of product changes, particularly when there are frequent releases. Consequently, software upgrades can be quite challenging.
Preconditions- the state before performing/running this use case	E.g., Multiple product releases exist, and an upgrade is anticipated.
Postconditions- the state when the use case finish	E.g., A document is generated with correct product changes in terms of functionalities and interfaces using a LLM chatbot.

Phase 2: Use case specification (approx. 40 minutes). Participants worked individually or in small groups to identify GenAI use cases from their own experiences using a structured template (see Table 3). Each individual or group filled out the template to document their use cases uniformly, capturing details such as the use case name, description, target users, challenges addressed, preconditions, and expected outcomes. The workflow for this phase:

1. Participants wrote down their use cases individually or collaboratively.
2. After documenting, participants placed their templates (or post-it notes summarizing their templates) on a shared whiteboard.
3. A round-robin discussion followed, during which participants presented their use cases one by one. This allowed the group to collectively review, clarify, and enhance each use case.

The workshops in Phase 2 were facilitated using guided prompts that asked participants to describe ongoing GenAI use cases, associated quality concerns, and any evaluation strategies currently applied. Workshops were conducted using shared online collaboration boards and real-time notes to document participant input. Two researchers independently reviewed the workshop notes and synthesized recurring patterns, including task types, quality attributes (e.g., accuracy, factuality), and metric use or absence.

The synthesized outputs from each company were then cross-compared to identify recurring themes across cases. These themes, particularly regarding domain-specific evaluation methods/techniques and perceived metric gaps, were used to design the semi-structured interview protocol for Phase 3. Interview questions were constructed to validate workshop findings, explore rationales behind metric use, and capture practices not raised in the workshops. This structure ensured continuity between study phases and allowed deeper investigation of observed practices.

Phase 3: Discussion (approx. 40 minutes). In this phase, the first author presented the preliminary grouping of use cases and invited participants to provide feedback on the clustering. The group reviewed the clusters and engaged in open discussions to refine the groupings, clarify specific use cases, and address questions.

The discussions were broad in scope, covering user interactions with GenAI applications (e.g., achieving specific tasks), developer-centric perspectives such as technical insights, and additional viewpoints like testing, specification, and managerial considerations. This multi-focus ensured that the workshops captured experiences from diverse stakeholders, including developers, managers, end users, and testers, while also addressing the broader context of GenAI applications. This phase facilitated a deeper understanding of the use cases, reduced potential misunderstandings, and allowed participants to contribute diverse perspectives from their professional roles.

Following the discussions, the first author summarized the key insights shared by participants and reviewed the clustering. The clusters that garnered the most feedback and discussion were highlighted. It is important to note that this observation reflects the use cases and application areas that participants found most relevant or engaging during the workshop. However, these clusters do not indicate objective importance or represent the full breadth of possible applications. The clustering and prioritization reflect the perspectives of the workshop’s participants based on their current roles and experiences, which could vary if the context or participant sample were changed/expanded. Thus, presenting a potential threat to both the internal and external validity of the study’s results, as further discussed in Section 6.

Finally, the workshop concluded by summarizing the key results and providing the participants with contact information to provide additional information.

3.3.2 Interview design

Following the workshops, we conducted semi-structured interviews with participants to gain further insights into quality measurement practices, the metrics used, and the challenges encountered in applying them.

The semi-structured interview guide was developed by the first three authors. Its design was based on established guidelines for qualitative research [34], and it was based on insights that emerged from our Phase 2 workshops. To improve construct validity, we conducted a pilot interview and refined the interview protocol for clarity and alignment with the study’s objectives. We also targeted multiple roles (e.g., developers, architects, managers, operations, and document engineers) to gather diverse perspectives on GenAI quality evaluation.

The guide consisted of thematic sections covering I) the participant’s role and involvement in GenAI adoption; II) current practices for evaluating GenAI outputs; III) challenges in metric usage. The complete interview guide is available on Figshare.

Each interview was divided into two phases: introduction and core questions.

Phase 1: Introduction (approx. 5-10 minutes). In the introduction, the interviewer outlined the purpose of the interview and clarified its goals. While the workshops aimed to gather and structure collective knowledge about use cases, the interviews were designed to acquire individual experiences, focusing on detailed explanations of metrics, evaluation methods, and challenges. This complementary focus was explained to participants. Participants were reminded of the confidentiality of their responses to encourage free and objective sharing of their experiences.

Phase 2: Core questions (approx. 30-40 minutes). The core of the interview consisted of open-ended questions that were structured into four key areas:

- Metrics and methods: Participants were asked to describe the specific metrics used to assess GenAI quality and the methods employed to measure each metric.
- Measurement techniques: Questions focused on how participants approached quality measurement, distinguishing between automated, human, and AI-assisted methods.
- Contextual factors: Participants provided details on how organizational and project-specific factors influenced their quality measurement processes.
- Challenges: Participants discussed any challenges encountered in evaluating GenAI quality, including technical and non-technical aspects.

In the end, we conducted 23 interviews across the three selected projects. The interviews were conducted in a mixed format, with onsite and online meetings, depending on participants’ availability and location. Each interview lasted between 43 and 55 minutes, allowing sufficient time to explore each topic without burdening the participants.

3.4 Step 4 - Data analysis

We conducted a thematic analysis [8] of the collected data to identify patterns and themes related to GenAI use cases, metrics, and challenges. An overview of the analysis process is presented in Figure 3.

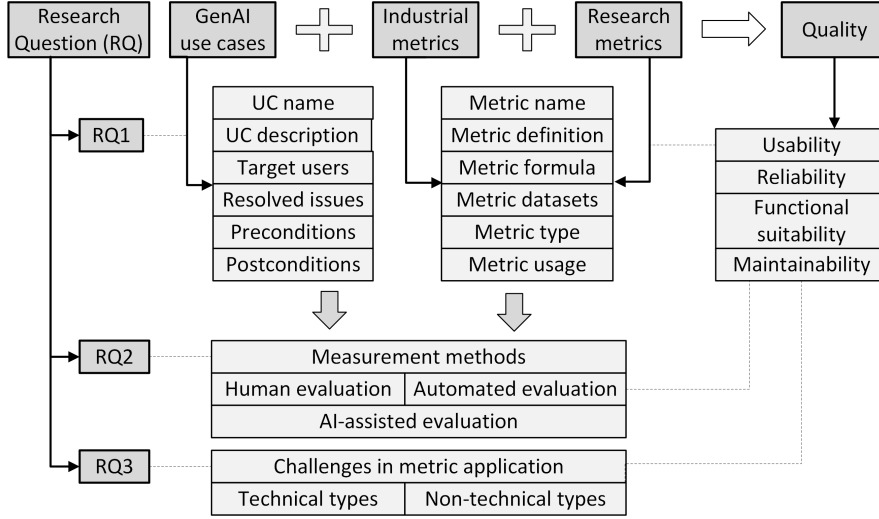


Fig. 3 Data synthesis based on collected data from the selected industrial projects

This analysis was performed according to the following steps:

1. *Create initial codes*: A starting set of codes was generated based on recurring concepts in the workshop notes and interview transcripts. The codes included terms related to the collected use cases (e.g., “trouble report analysis”), metrics (e.g., “accuracy”), and quality measurements (e.g., “automated evaluation”). The initial set was guided by our research questions and relevant phrases extracted directly from the data.
2. *Review extracted codes*: The codes were iteratively reviewed and refined to ensure that the codes reflected specific aspects of the data and avoided ambiguity. For example, a code initially labeled “accuracy” was refined to distinguish between “accuracy of document retrieval” and “accuracy of code review.”
3. *Connect and combine codes*: Related codes were clustered to form broader categories. For instance, codes related to “accuracy of outputs,” “correctness of outputs,” and “relevance to context” were grouped into a broader theme — “usability quality characteristics.” Similarly, “difficulties with the use of metrics” and “human evaluation challenges” were combined under “challenges in metric application.”

4. *Derive themes*: These categories were further synthesized into overarching themes that aligned with the study’s research questions. Thematic constructs such as “evaluation practices” and “metric usage” were developed to encapsulate key findings.

To strengthen internal validity and reduce researcher bias, data coding was performed collaboratively by three authors in iterative rounds. Disagreements were resolved through discussion, and key interpretations were reviewed in joint analysis sessions. Preliminary findings were also validated with participants through workshop feedback and cross-checking, supporting the interpretation’s reliability.

3.4.1 Experiment-driven metrics and mapping process

To incorporate research-based metrics into our analysis, we drew from a systematic snowballing literature review conducted in our previous research (Yu et al. [40]). The review study followed Wohlin et al. [37] guidelines and identified peer-reviewed metrics used to evaluate AI-generated outputs in software engineering contexts.

For clarity, ‘experiment-driven metric’ was defined as those identified in academic literature and developed for theoretical or experimental purposes. As illustrated in Table 4, metrics were classified as industrial if applied in any studied projects that were simple to compute and interpret within specific contexts. Experiment-driven metrics were often more complex, requiring standardized datasets, embeddings, or advanced tools for their computation.

Table 4 A classification of industrial and experiment-driven metrics

Classification aspect	Context-specific metric	Experiment-driven metric
Application context	Industrial settings: - System-specific quality evaluation - Development processes and artifacts (e.g., bug triage accuracy)	Research settings: - Lab-specific cases - Outside specific context (e.g., semantic similarity)
Data type	Domain-specific documents and codebases	Standardized/Sample data Synthetic data
Computation	Rule-based, interpretable metrics (e.g., accuracy of task assignment)	Embedding or lab-based scoring (e.g., FActScore)

The mapping was independently performed by the first two authors, who manually evaluated the contextual relevance of each experiment-driven metric to the GenAI use cases and associated quality characteristics. The results were then compared and refined collaboratively. Disagreements between authors were resolved through discussions, guided by a re-examination of the metric definitions and the industrial context. The final mapping was reviewed and confirmed by all authors to ensure consistency and validity. The mapping process is:

1. *Relevance screening*: Metrics identified from the literature were screened for relevance to the quality characteristics (e.g., Accuracy, Correctness, Relevance).

2. Contextual fit: Each metric was assessed for its applicability to specific use cases based on its original intended purpose and the type of output it evaluates (e.g., BERTScore for semantic similarity in document summarization, CodeBLEU for evaluating code generation).
3. Domain-expert feedback: The mapping was reviewed by workshop participants, who provided feedback on the practical feasibility of adopting these metrics in their projects. For example, participants suggested that some metrics, such as FActScore, could complement industrial practices by offering a fine-grained evaluation of factual consistency.

4 Results

This section presents our findings to answer the study’s research questions.

4.1 Results of RQ1 – What metrics are used to evaluate the quality of GenAI applications?

Firstly, we collected 14 industrial GenAI use cases presented in Table 5. These use cases are reported directly as they emerged from the workshops and interviews. Each row documents an existing use case, including its name, description, target users, issues to be resolved, preconditions for execution, and expected outcomes as postconditions. For instance, UC1 highlights how intelligent chatbots can retrieve relevant information from customer product documents. Similarly, UC3 demonstrates how GenAI applications integrated with GitLab can streamline code review processes by examining merge requests.

Table 5 Overview of collected industrial GenAI use cases. UC stands for Use Case.

ID	Use case name	Description	Target users	Issues to be resolved	Preconditions to run the use case	Postconditions of the use case
UC1	Intelligent ChatBot	Information retrieval against customer product documents.	Developers Architects Customers	Poor results of keyword searching against many documents.	Customer product information (CPI) documents are ready.	GenAI APPs retrieve the most relevant information from documents' embeddings based on user queries.
UC2	CodingBuddy	Coding assistants integrated in IDE	Developers	Developer productivity	Code repository are ready. IDE is used.	GenAI APPs suggest codes while developers are coding in IDEs.
UC3	GitlabReview	GenAI APP reviews Gitlab merge requests	Developers	Lead time for waiting merge request approvals	Integration to Gitlab and code repositories	GenAI APPs present, rank, and summarize code reviews.
UC4	Log classification	GenAI classifies log files.	Architects Developers	Grouping all reasons for failed functional testing jobs.	Log files are collected from failed functional tests.	GenAI APPs process log files and classify failures into groups.
UC5	Repo analyzer	GenAI assesses Gitlab repositories	Developers Architects	Root-cause analysis in a repository is challenging	Major issues have been reported or observed in a repo	GenAI APP analyzes codes, texts, files, and folders to suggest improvements.
UC6	DeveloperChatBot	Code generation for developer productivity.	Developers	Barriers of knowledge sharing	A knowledge base is required	LLM applications suggest relevant codes based on developers' queries.
UC7	Trouble report analysis	Analyze trouble reports to identify root-causes.	Developers Operations	Root-cause analysis of bugs/errors	Trouble reports contain detail logs of the run-time system	LLM applications suggest the most relevant root causes within the system.
UC8	Document ChatBot	Information retrieval against provided documents.	Developers Architects Testers Managers Customers	Poor results of keyword searching against many documents. Human effort of keyword searches.	The documents are accessible	LLM applications retrieve relevant and expected information from documents' embeddings based on user queries.
UC9	Development copilot	Coding assistants	Developers	Developer productivity	Existing codebases are accessible	LLM applications generate codes that are relevant to the provided contexts.
UC10	Document multi-agents	Document generation, summarization, and refinements.	Developers Architects Testers	Time and effort for document handling	GenAI multi-agent models	LLM agents process documents and generate expected outputs.
UC11	Intelligent code reviewer	GenAI APP reviews code changes in a codebase.	Developers	Leadtime for a proper code reviews	Extracting code changes and code context data are required	LLM APP presents, ranks, and summarizes code reviews.
UC12	AI for anomaly detection	AI-powered anomaly detection for CI logs	DevOps Developers	Failures in CI logs mix all types of errors.	Log files are accessible and captured.	Source information or original texts are cited in LLM generations.
UC13	Test case generation	Generate test cases from provided requirements	Developers Testers	Manual testing is time consuming. Maintaining tests is challenging	Requirements are prepared with detail information.	The data extracted from requirements are knowledge base are input for LLMs as contextualized embeddings to generate test cases.
UC14	Trouble report management	AI APP predicts links between TRs and components of a system	Managers Developers Architects	Lead-time is ~1.5 week while distributing TRs to the "Right" Dev teams	TR contains sufficient text description and logs from the run-time systems.	LLM APP suggests ranked components of a system linking to Dev teams.

Secondly, we extracted 28 metrics from the selected industrial projects by interviewing study participants. These metrics, including their associated datasets, definitions, and related use cases (UCs), are present in Table 6.

Each metric is categorized based on its evaluation focus, such as accuracy (Acc.), correctness (Corr.), alignment (Align.), relevance (Rel.), transparency (Tran.), completeness (Comp.), security (Sec.), and factuality (Fac.).

The table highlights the specific datasets used for each metric (e.g., requirement documents, codebases, system logs). For instance, ‘Acc.A’ measures accuracy using requirement documents and is linked to UC3 (GitLab reviews), while ‘Rel.A’ focuses on relevance using user manuals in UC10 (document multi-agents). This second finding illustrates the diversity of metrics and their applications that connect to the identified use cases.

Table 6 Collected metrics, datasets, definitions, and formulas from study participants. Note: Each metric is shown only in the use case(s) where it was explicitly reported by participants. While some metrics may conceptually apply to multiple use cases, we did not generalize beyond the empirical data.

Index	Metric ID	Datasets	Quality characteristics	UC ID
1	Acc.A	Requirement docs Source codes	Usability	UC3
2	Acc.B	Requirement docs	Usability	UC10
3	Acc.C	Codebases	Usability	UC11
4	Acc.D	Product component documents	Usability	UC14
5	Acc.E	System logs	Usability	UC7
6	Acc.F	CI build logs	Usability	UC12
7	Acc.G	Code repositories	Usability	UC2
8	Acc.H	Code repositories	Usability	UC9
9	Corr.A	SQL schema	Functional Suitability	UC3
10	Corr.B	Code repositories	Functional Suitability	UC5
11	Corr.C	Programming docs	Functional Suitability	UC6
12	Corr.D	Technical docs	Functional Suitability	UC8
13	Corr.E	Requirement docs	Functional Suitability	UC13
14	Align.A	Functional test logs	Functional Suitability	UC4
15	Align.B	CI logs	Functional Suitability	UC12
16	Align.C	Developer knowledge base	Reliability	UC8
17	Cons.A	Trouble report logs	Reliability	UC4
18	Rel.A	User manuals	Functional Suitability	UC10
19	Rel.B	Gitlab merge requests	Functional Suitability	UC11
20	Rel.C	Trouble reports	Reliability	UC7
21	Rel.D	Coding documents	Maintainability	UC2
22	Rel.E	Developer queries	Maintainability	UC9
23	Rel.F	Test cases	Maintainability	UC13
24	Tran.A	Trouble report logs	Usability	UC14
25	Comp.A	Code reviews	Reliability	UC7
26	Sec.A	System audit logs	Security	UC1
27	Fac.A	Product manuals	Usability	UC1
28	Fac.B	Product documents	Usability	UC6

Thirdly, we grouped the collected use cases and metrics into LLM application domains. Table 7 presents four primary application domains: *Document Generation*, *Data Analysis and Insights Generation*, *Customer Service ChatBot*, and *Code Generation*.

For each use case (e.g., UC3, UC10), the table provides the metrics currently used in the projects and suggests experiment-driven metrics that could enhance quality measurement. For instance, in the Document Generation domain, metrics like Accuracy (Acc.) and Correctness (Corr.) are applied, while research metrics such as METEOR and FActScore are proposed for further evaluation. Similarly, in Code Generation, metrics like Accuracy (Acc.) are complemented by suggestions like CodeBLEU and PASS@k. The table also shows the associated project names, linking the use cases to their industrial contexts.

Table 7 LLM application domain view of industrial use cases and metrics — the one that has been used or can be used.

LLM application domain	Use case ID	Metrics (used)	Research metrics (can be used)	Project name
Document generation	UC3	Acc. (Accuracy of reading XMLs, APIs, and requirements)	METEOR[2]	Project C
		Corr. (Correctness of API calls)	FActScore[23]	
	UC4	Align. (Alignment between LLM outputs and human expectations)	F1Score[19]	
		Cons. (Consistency of the LLM outputs towards the same or similar prompts)	FrugalScore[9]	Project A
	UC10	Acc. (Accuracy of text summarization)	BERTScore[41]	
		Rel. (Relevance of retrieved information from source documents)	YISI[22]	Project B
	UC11	Acc. (Accuracy of code review comments produced by LLM-based systems)	COMET[32]	
		Rel. (Relevance of generated code review reports)	CodeBLEU[33]	Project B
	UC14	Tran. (Transparency of trouble report (TR) generation)	MoverScore[43]	
		Acc. (Accuracy of distributing trouble reports to teams/individuals)	SARI[39]	
Data analysis and insights generation	UC5	Corr. (Correctness of generated insights)	SummEval[11]	Project C
		Comp. (Completeness of the outputs generated by LLM-based systems)	QAGS[36]	
	UC7	Rel. (Relevance of the generated TR analysis)	F1Score[19]	Project A
		Acc. (Accuracy of the generated TR analysis)	TRUE[13]	
	UC12	Align. (Alignment between the identified anomalies and human expectation)	N/A	Project B
		Acc. (Accuracy of the identified anomalies)	METEOR[2]	
Customer service ChatBot	UC1	Sec. (Security of system vulnerabilities)	N/A	Project C
		Fac. (Factuality of outputs produced by the ChatBot)	TRUE[13]	
			FActScore[23]	
	UC6	Corr. (Correctness of generated texts and citations)	BLEURT[35]	Project B
		Align. (Alignment of generated texts and source information)	N/A	
	UC8	Corr. (Correctness of chat content towards document embeddings)	MoverScore[43]	Project A
		Fac. (Factuality of generated chat content)	COMET[32]	
Code generation	UC2	Acc. (Accuracy of the generated codes)	PASS@k[15]	Project A
		Rel. (Relevance of generated codes)	CodeBLEU[33]	
	UC9	Corr. (Correctness of generated test cases)	Code_eval[6]	Project B
		Rel. (Relevance of generated tests)	PASS@k[15]	

This classification highlights the alignment between practical applications and academic insights, offering a structured view of GenAI’s role in industrial quality evaluation.

4.2 Results of RQ2 – What types of evaluation methods are used to measure the identified metrics for GenAI applications?

This section begins by detailing the usage of the metrics collected in RQ1, organized by domain as discussed in sub-sections 4.2.1 to 4.2.4. Following this, we describe the evaluation methods in sub-section 4.2.5, providing an overview of the approaches used in different contexts.

4.2.1 Document generation domain

The Document Generation domain includes industrial GenAI applications designed to produce, summarize, or refine documents based on user input or pre-existing data. We identified five use cases (UC3, UC4, UC10, UC11, and UC14) within this domain, focusing on different quality characteristics such as Usability, Reliability, and Functional Suitability. These use cases include code review, log classification, document analysis, and trouble report management.

Table 8 Metrics used in document generation domain.

Quality characteristic	Metric	Collected industrial use case
Usability	Acc. (Accuracy)	Create/improve documents (UC10) Code review comments (UC3,UC11) Distributed trouble reports (UC14)
	Tran. (Transparency)	Managing trouble report tickets (UC14)
Reliability	Cons. (Consistency)	Classified log groups (UC4)
Functional Suitability	Align. (Alignment)	Log classification (UC4)
	Rel. (Relevance)	Retrieved data from docs or codes (UC10,UC11)
	Corr.(Correctness)	Code generation (UC3)

Metrics that have been used in the document generation domain: Table 8 summarizes the metrics used for each quality characteristic. The metrics applied across the identified use cases include accuracy, alignment, consistency, relevance, and transparency. For instance, accuracy is critical in document summarization (UC10) to capture essential details, while alignment is essential in classification tasks (UC4) to ensure that outputs align with human expectations.

Practitioners’ insights: Study participants shared their insights into how GenAI applications improved their workflows and addressed specific pain points in the Document Generation domain. A common document generation workflow is illustrated in Fig. 4 based on study participants’ inputs. The workflow involves stages from document processing to GenAI-based generation, with most use cases relying on Retrieval-Augmented Generation (RAG) using proprietary

enterprise data. The identified context-specific metrics focus on evaluating the outputs of GenAI applications and often depend on domain experts' judgments.

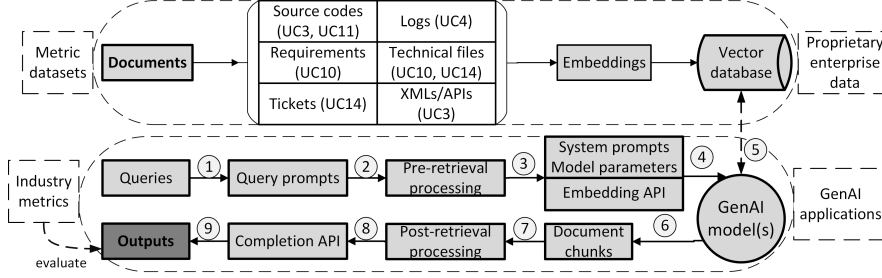


Fig. 4 Document generation general workflow based on study participants' inputs.

For **code reviews (UC3, UC11)**, several participants highlighted the impact of GenAI on code review processes. One senior developer noted how GenAI solutions reduced the time spent on reviewing complex SQL-related code changes, which often required domain-specific expertise: “Before adopting GenAI, reviewing SQL code was tedious and error-prone, especially when team members lacked expertise in database optimization.” Another feedback was, “With GenAI, the tool now flags potential issues, such as unnecessary joins or non-parameterized queries, which helps us address potential vulnerabilities and performance bottlenecks.” This insight reflects how GenAI enhances productivity by acting as an assistant, particularly in areas outside developers' immediate expertise. However, participants reported that “the outputs of GenAI are not always optimal and often require human refinement to ensure completeness and accuracy.” For example, when the top-ranked code suggestions fail to retrieve the most relevant or contextually accurate recommendations, critical issues may be overlooked.

Regarding **Log classification and troubleshooting (UC4, UC14)**, a lead software architect explained: “We generate thousands of log entries daily from automated tests, and identifying root causes used to be like searching for a needle in an ocean.” As stated by a developer, “GenAI's automated classification groups similar errors, such as ‘flaky test issues’ or ‘network failures,’ allowing us to prioritize critical problems for fact-based decisions.” However, challenges remain, particularly when logs contain ambiguous terms or abbreviations that GenAI struggles to interpret accurately. This sometimes leads to “wrong” classifications, requiring manual re-evaluation by domain experts.

In **document summarization and creation (UC10)** tasks, GenAI was employed to mitigate the burden of processing lengthy technical documents. A project manager shared: “Reviewing hundreds of pages of requirements or design documents was time-consuming and mentally exhausting.” GenAI pro-

duces structured summaries, which can assist engineers to quickly grasp the main content and focus on strategic decisions. However, some participants noted limitations, such as missing critical content when the GenAI model fails to identify the most relevant texts or keywords from the source documents, often due to incorrect ranking of retrieved data chunks.

Across these use cases, participants reported common themes while using GenAI for document generation. One recurring issue was “non-satisfaction” with outputs, where GenAI provided responses that were either too vague or overly detailed or failed to meet the required level of specificity. Another one was the system’s dependency on the quality and completeness of proprietary enterprise data. Missing data or poorly formatted input often led to errors, requiring manual intervention to correct or refine system outputs.

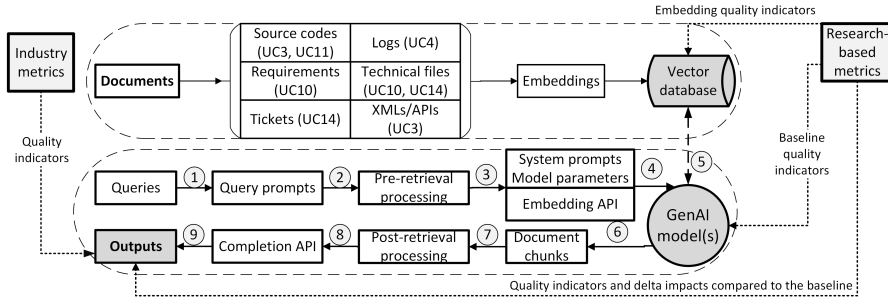


Fig. 5 Differences between industrial used metrics and experiment-driven metrics.

Experiment-driven metrics in this domain: Metrics, such as METEOR for semantic accuracy or F1Score for classification accuracy, often utilize standardized datasets and embeddings to evaluate theoretical improvements. As shown in Fig. 5, the metrics operate through a sequential process: they interact with datasets (e.g., embeddings), evaluate outputs generated by GenAI models, and compare these outputs against predefined baselines to assess their actual quality impact. Based on our discussions with participants, experiment-driven metrics can offer:

- A deeper understanding of GenAI application quality: Depending on the specific needs of the evaluation, experiment-driven metrics can provide more detailed insights into certain quality attributes (e.g., factuality or semantic alignment).
- A more automated approach for quality evaluation: These metrics often come with scripts and tools, which can streamline evaluations in controlled environments, reducing reliance on subjective judgments. However, the extent to which automation is feasible depends on the availability of the datasets and the alignment of the metrics with industrial contexts.

4.2.2 Data analysis and insight generation domain

The “Data analysis and insight generation” refers to a thematic grouping of the collected use cases where GenAI applications are employed to analyze data and produce actionable insights. These insights enable teams to make informed decisions based on patterns, trends, and contextual information extracted from complex datasets.

Table 9 Metrics used in data analysis and insight generation domain.

Quality characteristic	Metric	Identified industrial use case
Functional Suitability	Corr. (Correctness)	Analyzing GitLab Repositories (UC5)
Reliability	Comp. (Completeness)	Root-Cause Analysis (UC7)
Functional Suitability	Rel. (Relevance)	Detected anomalies (UC12)
Usability	Acc. (Accuracy)	Detecting Anomalies in CI Logs (UC12)

We identified three use cases in this domain (UC5, UC7, and UC12), which focus on improving productivity and accuracy in tasks such as code repository analysis, root-cause analysis, and anomaly detection. Table 9 summarizes the metrics that have been used to evaluate the accuracy, completeness, relevance, and correctness of the generated insights. For example, Completeness is crucial in root-cause analysis (UC7) to ensure all relevant patterns are identified, while Accuracy is key for anomaly detection (UC12) to catch issues impacting CI builds’ stability.

Practitioners’ insights: Study participants shared their insights on how GenAI applications have reduced the manual effort in the data analysis and insight generation domain. A general workflow is shown in Fig. 6 based on the collected data from participants. The process begins with uploading data from several sources, including codebases, logs, and tickets. The data undergoes querying and is processed with system prompts and model parameters before entering the GenAI models. The outputs generated by the GenAI model are then evaluated using the identified metrics.

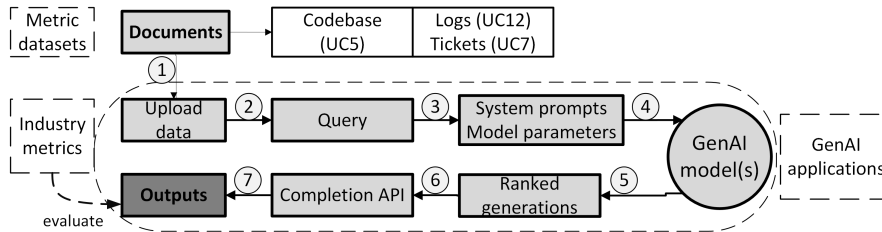


Fig. 6 A generic workflow for data analysis and insight generation using GenAI.

In **UC5** (analyzing gitLab repositories), a software architect highlighted how GenAI has transformed the process of understanding large codebases. “Analyzing dependencies and identifying technical debt was once a labor-intensive and error-prone task,” the architect explained. “With GenAI, we can quickly identify redundant modules and outdated dependencies, which could otherwise lead to security vulnerabilities.” However, participants noted a few challenges, such as cases where a GenAI application provided incorrect suggestions due to its inability to fully understand the highly customized codebases.

In **UC7** (root-cause analysis for trouble reports), an operations manager shared his experience of using a GenAI application to streamline the analysis of trouble reports. “Reviewing and diagnosing issues across complex, multi-component systems was challenging. GenAI’s ability to identify patterns and pinpoint root causes can reduce our troubleshooting time, especially for issues that span across different modules.” Despite this, a limitation frequently mentioned was that GenAI struggled with issues involving interdependencies between modules. The generated analysis was either incomplete or too generalized, necessitating deeper manual investigation.

For **UC12** (anomaly detection in CI logs), a DevOps engineer reported that “manually identifying critical errors in the CI environment was not easy. With GenAI models, we can catch issues earlier in the process, which not only reduces the time needed to fix critical build errors but also enhances CI jobs’ stability.” Meanwhile, several developers pointed out that GenAI’s anomaly detection sometimes produced false positives, requiring teams to correct them manually.

Challenges in industrial adoption: While these experiment-driven metrics offer a more granular evaluation, their adoption in industry often requires specialized knowledge, particularly in setting up the metrics and integrating them into existing workflows. Metrics like QAGS and BLEURT rely on embedding models, which may not be familiar to many industrial teams. While interpreting the results is generally straightforward, the primary challenge lies in configuring these metrics, such as the preparation of required datasets. This knowledge gap can limit their immediate applicability without prior training or expert guidance.

4.2.3 Customer service chatbot domain

This domain employs GenAI applications to enhance customer interaction by providing more context-aware responses. GenAI chatbots can handle a wide range of customer queries, from answering product-related questions to technical queries. The primary goal is to streamline customer support processes, reduce wait times, and improve customer satisfaction.

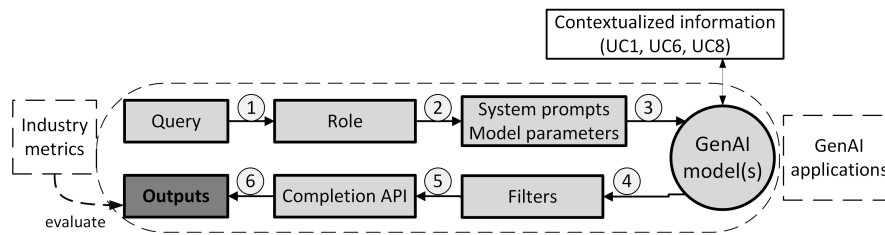
We identified three use cases within this domain: **UC1**, **UC6**, and **UC8**, focusing on two main quality characteristics: usability and reliability.

Table 10 Metrics that have been used in the customer service chatbot domain.

Quality characteristic	Metric	Identified Use Case
Usability	Corr. (Correctness)	Product support chatbot (UC1)
	Fac. (Factuality)	Developer chatbot for coding assistant (UC6)
Security	Sec. (Security)	Product operation (e.g., delete commands) (UC1)
Reliability	Align. (Alignment)	ChatBot for query knowledge base (UC8)

Context-specific metric: Table 10 shows the metrics that have been utilized for evaluating Usability, Security, and Reliability of a software system. For example, Factuality is critical for the developer ChatBot (UC6) to ensure that coding support provided by GenAI is technically sound, while Security is crucial for the Intelligent ChatBot (UC1) to safeguard sensitive information during user interactions.

Practitioners' insights: Study participants described a common workflow while employing GenAI for chatbot services. The workflow is illustrated in Fig. 7. The main differences compared to the other application domains are: I) the chatbot's role specification is required; II) contextualized information through embeddings or prompts is needed.

**Fig. 7** A common workflow for customer service chatbots.

In **UC1** (chatbot for product information support), a product support manager noted that the GenAI chatbot reduced the volume of routine queries handled by operation teams. As the manager explained, “Since implementing the GenAI-powered chatbot, we have been able to handle a larger number of queries securely, reducing the workload on our teams and improving response times.” This benefit was echoed in **UC6**, where a software engineer observed that the GenAI chatbot enabled developers to access relevant code snippets and best practices directly within their workflow, accelerating troubleshooting and implementation. Similarly, in **UC8**, a technical writer highlighted the chatbot’s ability to retrieve information from large document collections, which improved user satisfaction and reduced search times. In short, these use cases illustrate how GenAI chatbots can streamline operations, enhance productivity, and make information more accessible.

However, study participants reported several limitations that hindered the full potential of GenAI-based chatbots. A major challenge across all use cases was maintaining the accuracy and factuality of GenAI system outputs. For example, in UC1 and UC8, incorrect information retrieval or outdated content led to user frustration and increased follow-up queries, indicating a need for regular data curation and model updates. In UC6, the Developer ChatBot occasionally provided code snippets incompatible with the other dependencies, requiring developers to rework existing code.

Challenges in industrial adoption: It is important to note that many metrics, such as FActScore and TRUE, require computational resources that may hinder real-time responsiveness, which is critical for applications like customer service chatbots or some decision-making systems. The complexity of implementing metrics like MoverScore, which relies on extra AI models, poses challenges for industrial teams due to the increased costs, required additional computational resources (e.g., GPUs).

4.2.4 Code generation domain

The Code Generation domain uses GenAI applications to assist developers in generating code snippets, fixing bugs, and providing code suggestions. We found three use cases (UC2, UC9, and UC13) in this domain, and each UC evaluated different system quality characteristics, such as Usability, Reliability, and Maintainability.

Table 11 Metrics used in the code generation domain.

Quality characteristic	Metric	Identified Use Case
Usability	Acc. (Accuracy)	Coding Assistants in IDE (UC2,9)
Reliability	Corr. (Correctness)	Contextualized test case generation (UC13)
Maintainability	Rel.(Relevance)	Code Generation (UC2,9,13)

Metrics: Table 11 presents the quality characteristics, used metrics, and the identified use cases. For example, Acc. (Accuracy) is essential for GenAI-based coding assistants to ensure the generated code is accurate, while Corr. (Correctness) ensures reliability in contextually generated code for complex systems.

Practitioners' insights: The inputs from study participants enabled us to outline a general workflow illustrating how GenAI models were utilized across the identified use cases. As shown in Fig. 8, Integrated Development Environment (IDE) APIs served as the primary interfaces through which engineers interacted with GenAI models for code generation. While the APIs provide the necessary integration points, the scalability of these solutions primarily depends on the performance of the APIs, the underlying models queried through them,

and the hardware infrastructure supporting those models. Factors such as response times, concurrency handling, and the computational resources available to the models are critical in maintaining the quality of generated code and ensuring support for a large number of connected developers.

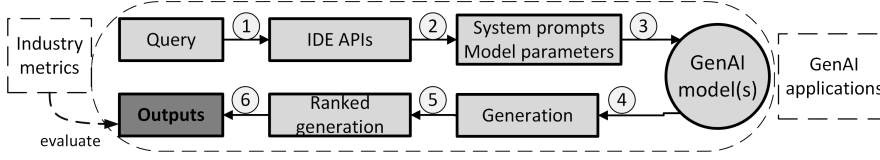


Fig. 8 A generic workflow for GenAI code generation based on participants' inputs.

An observation among developers was the ability of GenAI tools to help them ‘stay in the zone’ while coding. As one developer in UC2 mentioned: “Having an AI coding tool directly in the IDE means I do not have to break my ongoing work to search for code solutions online.” This ability to provide quick, contextual suggestions kept developers focused, accelerating task completion by integrating help directly where they work.

GenAI’s context-awareness was valuable when generating code that needed to fit into complex systems. A lead developer in UC9 praised Codeium’s ability¹ for producing code that aligns with the existing source files: “The contextualized code suggestions mean we’re not just getting code, but code that fits into our existing class/function.” This alignment may reduce the time spent on rework, making GenAI tools useful for implementing new features within established codebases. However, study participants reported that they occasionally fell short in handling more complex tasks, requiring developers to step in and make manual adjustments.

In testing, testers noted that manually creating test cases for each scenario was often a bottleneck. As one participant observed, “With GenAI, we generate test suites in a fraction of the time, which allows us to focus more on exploratory testing.” This improved test coverage for simple scenarios, but GenAI often missed critical edge cases that required a deeper understanding of specific requirements. As a result, human oversight is required to ensure that critical test scenarios are covered.

Challenges in adopting experiment-driven metrics: No major blockers have been reported that prevent the use of these experiment-driven metrics in industrial projects. However, one potential reason is that industrial engineers may simply be unaware of these metrics, indicating a need for closer collaboration between academia and industry. Additionally, industrial teams may require training to incorporate new metrics.

¹ <https://codeium.com>

4.2.5 Types of evaluation methods

Our investigation of the collected industrial GenAI use cases revealed that different metrics require different levels of domain-experts’ involvement and automation, depending on the complexity of the task and industrial contexts. Table 12 summarizes three types of evaluation methods, such as human, automated, and AI-assisted evaluations, to group the identified context-specific metrics.

Human evaluations are often necessary for tasks that require subjective judgment, contextual understanding, or expertise that cannot be easily codified into rules. However, the evaluation can be time-consuming and inconsistent across evaluators. On the other hand, **automated evaluations** enable faster, scalable assessments that can handle large volumes of data, but may lack the ability to understand complex scenarios. The emergence of **AI-assisted evaluations** bridges this gap by using AI models to assist or complement human and automated evaluations. These models can provide analyses that are more detailed and contextualized compared to simpler metrics or traditional human evaluations, such as contextual relevance scoring.

Regarding the metrics in the identified evaluation methods, **Accuracy** (Acc.) and **Correctness** (Corr.) show a balanced reliance on both *human* and *automated* evaluation, indicating that while automated checks can handle various validations, manual review is still essential for verifying the outputs.

Table 12 Evaluation types for the identified metrics.

Metric	Manual Eval.	Automated Eval.	AI-Assisted Eval.
Acc.	✓(manual review)	✓(automated checks)	✗
Corr.	✓(manual verification)	✓(automated validation)	✗
Align.	✓(expert assessment)	✓(semantic analysis)	✓(AI models)
Comp.	✓(manual check)	✓(automated checks)	✗
Rel.	✓(subjective review)	✗	✓(AI scoring)
Fac.	✓(fact-checking)	✗	✓(AI verification)
Sec.	✓(security review)	✓(vulnerability scan)	✗
Tran.	✓(clarity review)	✗	✗

Relevance (Rel.) and **Factuality** (Fac.) metrics primarily benefit from *AI-assisted* evaluation alongside human oversight. For instance, AI models can provide contextual relevance scores or fact-checking support, but final decisions often need to be reviewed by humans to ensure reliability.

Conversely, **Security** (Sec.) emphasizes the importance of *automated* evaluation, such as vulnerability scanning, while still requiring manual security reviews to catch context-specific issues that automated tools might miss. However, **Transparency** (Tran.) relies solely on human evaluation, reflecting that clarity and readability are mainly judged by human reviewers, given their subjective nature.

In summary, human, automated, and AI-assisted evaluation approaches can be employed while using industrial and experiment-driven metrics to mea-

sure the quality of GenAI-based systems. The practical insights and challenges highlighted in this study should be considered to ensure successful implementation in industrial applications.

4.3 Results of RQ3 – What challenges are encountered in applying metrics to evaluate the quality of GenAI applications?

As stated by participants from the selected projects, with the increasing deployment of GenAI, scalable evaluation methods are critical. Challenges such as automated output verification and managing context can hinder scalability. As stated by a developer in Project C, *“Manual checking is okay at first, but it can hardly scale when we deal with thousands of prompts per day.”*

Managing context refers to ensuring that the evaluation metrics account for the specific requirements and dependencies of the assessed use cases. For example, in code generation, the relevance of generated code snippets depends on their compatibility with existing frameworks or system architecture. A lead developer reported: *“A code snippet might work in isolation, but it would be useless if it does not fit our current technology stack.”*

Another issue raised was the interpretability of metrics. As mentioned by an R&D manager, *“the metric says 0.83 accuracy, but what does that mean for my team or product? We cannot act on that.”* Participants also highlighted a lack of awareness or access to advanced research metrics. For instance, a senior architect shared: *“I was not even aware of some of these evaluation tools until your interview. We mostly rely on internal manual verification.”*

Table 13 Challenges related to quality evaluation with metrics. MC refers to metric-related challenges.

ID	Challenge name	Description
MC1	Chunk Size for Retrieval	Choosing appropriate chunk sizes can impact the quality of GenAI outputs. Smaller chunks may improve precision but lose context, while larger chunks retain context but may dilute relevance. Balancing these factors is a technical challenge noted by Regenwetter et al. [31]
MC2	Human Output Verification	Manual review is essential for assessing nuanced aspects like relevance and factuality, but it can be time-consuming, inconsistent, and difficult to scale, as highlighted in Kenthapadi et al. [18]
MC3	Output Consistency	Achieving consistent responses for similar prompts is challenging. Minor variations in input phrasing can lead to different outputs, complicating the assessment of GenAI system reliability. This is a well-documented issue in GenAI systems [17]
MC4	Context Embeddings	Determining which context data to embed and how to do it effectively is crucial for improving GenAI outputs. Ineffective embeddings can lead to incorrect or irrelevant responses, as explored in Regenwetter et al. [31]
MC5	Metric Interpretability	Even when metrics are effectively implemented, their scores must be clear and interpretable for stakeholders. Ensuring metrics provide actionable insights rather than just numerical values is essential.

As illustrated in Table 13, five main challenges have been identified. These challenges align with prior research findings, which have explored issues like chunk size optimization and context embeddings in GenAI systems. For example, Regenwetter et al.[31] discuss the trade-offs in chunk sizes for retrieval,

while Jiang et al.[17] highlight the difficulty in achieving output consistency. However, MC5 (Metric Interpretability) emerged as a distinct concern in the industrial evaluations conducted in this study, emphasizing the importance of actionable metrics tailored to practical use.

5 Implications for practice

Based on the findings from our multi-case study, we highlight applicable quality aspects, recommend metrics, and suggest steps to apply them in four identified SE domains. We intend to help practitioners identify entry points for metric adoption and build progressively toward more structured quality evaluation practices for GenAI applications.

5.1 Document generation domain

Document generation can be used for summarizing requirements, generating implementation proposals, or producing compliance reports in software development. The quality of outputs in this domain is often judged by practitioners based on accuracy, completeness, and factual correctness, but a human-in-the-loop is needed. Based on our findings, we suggest the following metrics and practices.

Table 14 Suggested metrics and practices for document generation.

Quality aspects	Metric	Steps to apply in practice
Accuracy	Task-specific correctness rate (rule-based)	Step 1: Define required content (e.g., top 3 changes). Step 2: Count captured vs. missed or hallucinated items. Step 3: Use checklists or templates for structured review.
Relevance	BERTScore (embedding-based semantic similarity)	Step 1: Compare GenAI output with cited examples. Step 2: Set threshold ranges based on task type. Step 3: Apply in batch mode for multiple summaries.
Factuality	FActScore (embedding and entity-level matching)	Step 1: Apply to high-stakes content (e.g., compliance documents). Step 2: Automate comparison with trusted source documents if feasible. Step 3: Add manual review for edge cases.
Alignment with expectations	Reviewer scorecard (human checklist or rating)	Step 1: Ask stakeholders to rate clarity, coverage, and usefulness. Step 2: Use Likert-scale or task-based scoring forms. Step 3: Collect feedback to improve prompting and review flow.

Practitioners are encouraged to embed these checks into their current workflows. Lightweight rule-based metrics can serve as first-pass filters. Embedding-based metrics like BERTScore or FActScore can be used selectively, where precision, traceability, and quality assurance are critical.

5.2 Data analysis and insight generation

For analyzing software artifacts (e.g., logs, error reports, and historical issues), GenAI can be used to generate actionable insights or identify anomalies. Key quality characteristics for these tasks include correctness of insights, relevance, traceability to source data, and completeness. The following metrics and practices are recommended based on our findings.

Table 15 Suggested metrics and practices for data analysis and insight generation.

Quality aspects	Metric	Steps to apply in practice
Correctness	Issue classification accuracy (rule-based or supervised)	Step 1: Define labels for known categories (e.g., bug, enhancement). Step 2: Calculate accuracy or F1-score over labeled test sets. Step 3: Use a confusion matrix to diagnose misclassifications.
Relevance	Precision (ranking-based relevance metric)	Step 1: Use feedback from domain-experts to label top-k outputs as relevant or not. Step 2: Compute relevance proportion within top-k (e.g., top 3 results). Step 3: Optimize GenAI prompt or parameters for top-k precision.
Traceability	Reference coverage rate (rule-based)	Step 1: Check if generated insights include references to correct artifacts (e.g., logs). Step 2: Count correct vs. missing trace links. Step 3: Apply rule-based extraction for audit traceability.
Completeness	Insight completeness score (review-based)	Step 1: Define a checklist of expected insight components. Step 2: Manually score the presence or absence of components. Step 3: Use sample-based audits for scaling to larger datasets.

Practitioners should start with human validation of GenAI-generated insights and then integrate rule-based metrics or feedback scoring to gradually scale evaluation.

5.3 Customer service

GenAI-based support agents can assist users with troubleshooting and technical questions. Quality evaluation in this domain accounts for correctness, factual reliability, and information safety. Based on our findings, we recommend the following metrics and practices.

Table 16 Suggested metrics and practices for customer service.

Quality aspects	Metric	Steps to apply in practice
Correctness	Response accuracy (manual or rule-based)	Step 1: Define expected correct answers for standard queries. Step 2: Evaluate response correctness against known responses. Step 3: Track accuracy across different interaction types.
Factuality	FactScore (embedding & entity-level matching)	Step 1: Use for validating GenAI answers against verified knowledge bases. Step 2: Apply selectively for high-impact or regulated topics. Step 3: Log low-scoring responses for audit and improvements.
Role alignment	Role classifier score (supervised or LLM-based)	Step 1: Use classifiers to assess sentiment and role. Step 2: Align role to context (e.g., empathetic for error messages). Step 3: Integrate with QA workflows for flagged conversations.
Information safety	Sensitive content detection (rule-based or LLM-based)	Step 1: Use regex or LLM to detect leakage of private or harmful information. Step 2: Evaluate model responses against policy checklists. Step 3: Block or alert on risky responses in real time.

GenAI chatbots require quality controls that extend beyond functional correctness to include safety, trustworthiness, and user experience. It is important for practitioners to filter sensitive content and flag hallucinations using multiple question-answering metrics.

5.4 Code generation

GenAI has been used to support developers in writing, refactoring, reviewing, and documenting code. In this domain, evaluation focuses on accuracy, relevance to intent, functional validity, and maintainability. Based on our findings, we recommend the following metrics and practices.

Table 17 Suggested metrics and practices for code generation.

Quality aspects	Metric	Steps to apply in practice
Accuracy	Unit test pass rate (execution-based)	Step 1: Run the generated code against predefined unit tests. Step 2: Count total passed vs. failed cases. Step 3: Use as a gating metric for GenAI output acceptance.
Relevance	Intent alignment score (prompt-task match)	Step 1: Define intended functionality from prompts. Step 2: Review if the generated code aligns with the functional intent. Step 3: Use developer feedback to refine the scoring scale.
Functionality	CodeBLEU (embedding-based syntax & semantics)	Step 1: Compare the generated code with the reference implementation. Step 2: Use CodeBLEU to evaluate syntax and structure. Step 3: Combine with static analysis for a holistic view.
Maintainability	Cyclomatic complexity (static analysis)	Step 1: Measure the complexity of the generated code using analysis tools (e.g., PMD). Step 2: Set internal complexity thresholds by team/project. Step 3: Flag overly complex output for refactoring.

For GenAI-assisted coding, evaluation can be embedded directly into developers’ Integrated Development Environments (IDEs). Embedding-based metrics like CodeBLEU can complement human reviews.

6 Validity threats

We structure our discussion of validity threats following the guidelines by Runeson and Höst’s [34] four categories: internal, external, construct, and reliability validity. Mitigation strategies applied during the study design and execution are described in Section 3.

6.1 Internal validity

One potential threat is the distribution of participants across the selected projects. Participants from some roles, such as operation engineers or document engineers, may concentrate more on one or two projects. This uneven distribution could lead to some projects having more or fewer practices and challenges documented compared to others, potentially biasing the results.

To mitigate this potential threat, the data analysis [27] accounted for variations in project contexts, ensuring that insights from projects with more

participants were cross-verified against those with fewer participants. Representativeness was determined based on the diversity of roles and the relevance of the GenAI use cases within each project context, rather than simply the frequency of GenAI application.

6.2 External validity

The study focuses on three specific organizations, which cannot represent the full range of industries or geographical regions that use GenAI. Although it may be challenging to include all industries, selecting a diverse set of companies can enhance representativeness. Additionally, the metrics identified and discussed in this study reflect practices observed in these specific organizations and may not generalize to all industrial contexts. Future research is expected to address this limitation by broadening the scope of case studies to include a more diverse range of industries and application domains.

It is important to note that this study does not evaluate GenAI models themselves (e.g., architecture, licensing, cost-performance trade-offs, or infrastructure dependencies). Such model-level factors are crucial for system-level decisions but are beyond the scope of this work. Our focus remained on evaluating the output and quality characteristics of GenAI applications in practice, as experienced by industrial practitioners. Benchmarking GenAI models has been extensively studied elsewhere and complements but does not overlap with our objectives.

6.3 Construct validity

There may be differences in how certain metrics (e.g., accuracy, relevance) are defined and understood in research versus industrial contexts. This could lead to misunderstandings in data interpretation. We mitigated this threat by clarifying definitions during workshops and interviews, ensuring that all participants had a shared understanding of key concepts.

The use of self-reported data through interviews and workshops could introduce bias. Participants might overstate successes or underreport challenges. To address this, we assured participants of anonymity and encouraged honest and open discussions.

6.4 Conclusion validity

Given the exploratory nature of a multi-case study, there's a risk of over-interpreting qualitative findings. We ensured that our conclusions were grounded in the data by providing direct participant quotes and following a systematic coding process.

As this study is based on qualitative data, the findings may lack statistical generalizability. We aimed to provide insights into the experiences of practitioners. We recommend that future research include quantitative methods to complement and validate these findings.

7 Discussions

7.1 How much accuracy is enough?

While existing metrics provide ways to quantify the accuracy of AI-generated outputs, our findings suggest that achieving satisfactory levels of accuracy remains a challenge in many industrial applications. Participants often expressed concerns about the impact of inaccuracies on trust and user satisfaction. One participant highlighted that “a 10% inaccuracy in AI-generated outputs could lead to a permanent loss of customer trust.” While this statement highlights the critical role of precision in maintaining user confidence, it also underscores the importance of metrics in providing insights into system correctness. Metrics with high precision enable practitioners to understand and quantify the limitations of GenAI system outputs, guiding iterative improvements to the system. Without such metrics, identifying and addressing gaps in system correctness would largely rely on guesswork.

Another participant offered a broader perspective, suggesting that “Instead of focusing solely on accuracy, we should consider the tangible improvements GenAI tools bring to our workflow.” This view supports the importance of metrics beyond measuring accuracy—they serve as tools to evaluate incremental improvements and track the evolving performance of GenAI systems. Metrics provide the necessary data to establish thresholds for acceptable performance, which are critical for balancing improvement against the cost of inaccuracies. For example, identifying an acceptable accuracy threshold through metrics can help stakeholders decide when the system’s outputs are sufficient for deployment.

For less accuracy-sensitive domains, metrics can play an equally important role. Measuring improvements over time, such as tracking trends in accuracy gains or reductions in error rates, allows teams to monitor progress and make informed decisions. “What threshold of improvement or accuracy gain will satisfy end users?” becomes a question that metrics can help answer by providing more objective data.

7.2 Challenges of contextual understanding

One of our identified barriers to GenAI adoption in software engineering is the mismatch between output quality and contextual expectations. Our findings highlight a growing concern about whether the generated content aligns with task-specific and organizational context. For practitioners, it means that

teams cannot simply “drop in” a GenAI model and expect useful results. Instead, GenAI output evaluation needs to incorporate local standards — such as terminology, structure, or procedural conventions — which are not captured by many metrics. Our findings confirm the challenges raised in Kenthapadi et al. [18] about GenAI’s lack of grounding in enterprise-specific knowledge.

Practitioners spend manual effort to adapt GenAI outputs to their domain, such as embedding company-specific terminology into prompts, fine-tuning the information retrieval process, or involving human reviewers to validate the results. However, current evaluation tools and metrics do not fully support these needs. To address this gap, future metrics should cover: *context awareness* — able to validate whether an output respects domain constraints, and *configurable* — allowing teams to encode domain-specific rules.

For metric adaptation, our findings uncover that evaluation scores are useful when adjusted to domain expectations, which requires domain-expert feedback and custom thresholds. Practitioners should treat contextual evaluation not as a static task, but as an iterative process.

7.3 Bridging research and practice in metric usage

Our study shows a difference between experiment-driven and context-specific metrics. In academic research, experiment-driven metrics like BLEU, ROUGE, BERTScore, or FActScore are commonly used. These metrics offer evaluations of aspects like semantic similarity and factual accuracy. However, they rely on online datasets and reference outputs that are often unavailable or impractical in industrial settings.

On the other hand, practitioners in industry tend to use context-specific metrics grounded in artifacts used in their development, such as test cases, bug reports, or requirement documents. Using these artifacts, the metrics are easier to compute, interpret, and act upon, especially in domains with time or resource constraints.

Despite this divergence, the two groups of metrics should not be seen as mutually exclusive. Several metrics in academia, including FActScore or BERTScore, could be adapted to industrial use. For example, these metrics could be adopted to assess factuality in generated documentation, while rule-based systems or expert review layers capture domain-specific correctness. Moreover, industry teams may benefit from understanding the methodological rigor of academic metrics, especially for tasks like summarization, log interpretation, or code review, where context-aware accuracy is difficult to measure. Future research may explore how these two metric categories can be translated or hybridized for use in enterprise-grade GenAI systems.

8 Conclusions

This study presents 28 metrics, three evaluation approaches, and five challenges for the quality measurement of Generative AI (GenAI) applications

from 14 industrial GenAI use cases spanning domains such as document generation, data analysis, customer service chatbots, and code generation.

Study results highlight the importance of a multi-faceted approach to quality measurement for GenAI applications. For example, a combination of human, automated, and AI-assisted evaluation while using metrics. Metrics such as accuracy, relevance, alignment, correctness, transparency, completeness, security, and consistency are helpful for evaluating GenAI system outputs. However, traditional metrics alone are insufficient. Companies need to incorporate multiple/combined metrics that measure system qualities in specific industrial contexts. For instance, developers can leverage these metrics to answer critical questions, like “Are we making enough progress?” or “Is the application ‘good enough’ to be deployed?” Managers can gain a better understanding of an application’s quality during the development cycle, enabling informed decisions about deployment readiness.

Moreover, some metrics in research studies may not fully align with the specific requirements of industry applications. Industry practitioners prioritize metrics that are interpretable, actionable, and adaptable to domain-specific contexts. The identified challenges, such as metric interpretability and the use of context embeddings, reinforce the practitioners’ preferences.

In short, as GenAI has been integrated into industrial processes, practical quality measurement is needed. By combining both academic and context-specific metrics, organizations can improve the quality measurement of GenAI applications that reflect theoretical rigor and align with real-world applications’ demands.

For future work, building upon this study, a quality model could be developed by exploring broader industrial contexts. Such a model would synthesize quality characteristics, metrics, and software development domains to provide guidance on what to measure, when to measure, and how to achieve high quality in GenAI-supported development.

Acknowledgements We acknowledge support from the KKS Foundation through S.E.R.T. Research Profile Project (research profile grant 20180010) at Blekinge Institute of Technology.

Declarations

Conflict of interest: The authors declared that they have no competing interests that are relevant to the content of this article. I would like to declare on behalf of my co-authors that the work described was original research that has not been published previously and is not under consideration for publication elsewhere. All the authors listed have approved the manuscript.

Funding: This work was supported by the KKS Foundation (Grant number: 20180010).

Data Availability Statement: The data supporting this study are available on Figshare, including a semi-structured interview guide, participants’ demographics and work roles, and industrial metrics’ names, definitions, formulas, and datasets. These materials are provided to enhance the study’s transparency. The raw interview transcripts remain confidential because participants explicitly requested it, considering the risk of re-identification.

Author contributions: All authors contributed to the study’s conception and design. Material preparation, data collection, and analysis were performed by Liang Yu, Emil Alégroth, and Panagiota Chatzipetrou. The first draft of the manuscript was written by Liang Yu, and all authors commented on previous versions of the manuscript. All authors read and approved the final manuscript.

References

1. Allow, J., Neustadt, I.: Uml 2 and the unified process. Reading: Addison Wesley (1999)
2. Banerjee, S., Lavie, A.: Meteor: An automatic metric for mt evaluation with improved correlation with human judgments. In: Proceedings of the acl workshop on intrinsic and extrinsic evaluation measures for machine translation and/or summarization, pp. 65–72 (2005)
3. Bommasani, R., Hudson, D.A., Adeli, E., Altman, R., Arora, S., von Arx, S., Bernstein, M.S., Bohg, J., Bosselut, A., Brunskill, E., et al.: On the opportunities and risks of foundation models. arXiv preprint arXiv:2108.07258 (2021)
4. Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al.: Language models are few-shot learners. Advances in neural information processing systems **33**, 1877–1901 (2020)
5. Celikyilmaz, A., Clark, E., Gao, J.: Evaluation of text generation: A survey. arXiv preprint arXiv:2006.14799 (2020)
6. Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P.D.O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al.: Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 (2021)
7. Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H.W., Sutton, C., Gehrmann, S., et al.: Palm: Scaling language modeling with pathways. Journal of Machine Learning Research **24**(240), 1–113 (2023)
8. Cruzes, D.S., Dyba, T.: Recommended steps for thematic synthesis in software engineering. In: 2011 International Symposium on Empirical Software Engineering and Measurement, pp. 275–284. IEEE (2011)
9. Eddine, M.K., Shang, G., Tixier, A.J.P., Vazirgiannis, M.: Frugalscore: Learning cheaper, lighter and faster evaluation metrics for automatic text generation. arXiv preprint arXiv:2110.08559 (2021)
10. Esposito, M., Li, X., Moreschini, S., Ahmad, N., Cerny, T., Vaidhyanathan, K., Lenarduzzi, V., Taibi, D.: Generative ai for software architecture. applications, trends, challenges, and future directions. arXiv preprint arXiv:2503.13310 (2025)
11. Fabbri, A.R., Kryściński, W., McCann, B., Xiong, C., Socher, R., Radev, D.: Summeval: Re-evaluating summarization evaluation. Transactions of the Association for Computational Linguistics **9**, 391–409 (2021)
12. Feldt, R., de Oliveira Neto, F.G., Torkar, R.: Ways of applying artificial intelligence in software engineering. In: Proceedings of the 6th International Workshop on Realizing Artificial Intelligence Synergies in Software Engineering, pp. 35–41 (2018)
13. Honovich, O., Aharoni, R., Herzig, J., Taitelbaum, H., Kukliansy, D., Cohen, V., Scialom, T., Szpektor, I., Hassidim, A., Matias, Y.: True: Re-evaluating factual consistency evaluation. arXiv preprint arXiv:2204.04991 (2022)

14. Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., Wang, H.: Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology* (2023)
15. Huang, D., Bu, Q., Zhang, J.M., Luck, M., Cui, H.: Agentcoder: Multi-agent-based code generation with iterative testing and optimisation. *arXiv preprint arXiv:2312.13010* (2023)
16. ISO/IEC-25023: Systems and Software Engineering: Systems and Software Quality Requirements and Evaluation (SQuARE): Measurement of System and Software Product Quality. ISO (2016)
17. Jiang, D., Ku, M., Li, T., Ni, Y., Sun, S., Fan, R., Chen, W.: Genai arena: An open evaluation platform for generative models. *arXiv preprint arXiv:2406.04485* (2024)
18. Kenthapadi, K., Sameki, M., Taly, A.: Grounding and evaluation for large language models: Practical challenges and lessons learned (survey). In: *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 6523–6533 (2024)
19. Kynkäänniemi, T., Karras, T., Laine, S., Lehtinen, J., Aila, T.: Improved precision and recall metric for assessing generative models. *Advances in neural information processing systems* **32** (2019)
20. Lago, P., Runeson, P., Song, Q., Verdecchia, R.: Threats to validity in software engineering—hypocritical paper section or essential analysis? In: *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, pp. 314–324 (2024)
21. Lin, C.Y.: Rouge: A package for automatic evaluation of summaries. In: *Text summarization branches out*, pp. 74–81 (2004)
22. Lo, C.k.: Yisi-a unified semantic mt quality evaluation and estimation metric for languages with different levels of available resources. In: *Proceedings of the Fourth Conference on Machine Translation (Volume 2: Shared Task Papers, Day 1)*, pp. 507–513 (2019)
23. Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W.t., Koh, P.W., Iyyer, M., Zettlemoyer, L., Hajishirzi, H.: Factscore: Fine-grained atomic evaluation of factual precision in long form text generation. *arXiv preprint arXiv:2305.14251* (2023)
24. Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I.D., Gebru, T.: Model cards for model reporting. In: *Proceedings of the conference on fairness, accountability, and transparency*, pp. 220–229 (2019)
25. Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al.: Training language models to follow instructions with human feedback. *Advances in neural information processing systems* **35**, 27730–27744 (2022)
26. Papineni, K., Roukos, S., Ward, T., Zhu, W.J.: Bleu: a method for automatic evaluation of machine translation. In: *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, pp. 311–318 (2002)
27. Patton Quinn, M.: *Qualitative research & evaluation methods* (2002)
28. Petersen, K., Wohlin, C.: Context in industrial software engineering research. In: *2009 3rd International Symposium on Empirical Software Engineering and Measurement*, pp. 401–404. IEEE (2009)
29. Petersen, K., Wohlin, C.: The effect of moving from a plan-driven to an incremental software development approach with agile practices: An industrial case study. *Empirical Software Engineering* **15**, 654–693 (2010)
30. Raji, I.D., Smart, A., White, R.N., Mitchell, M., Gebru, T., Hutchinson, B., Smith-Loud, J., Theron, D., Barnes, P.: Closing the ai accountability gap: Defining an end-to-end framework for internal algorithmic auditing. In: *Proceedings of the 2020 conference on fairness, accountability, and transparency*, pp. 33–44 (2020)
31. Regenwetter, L., Srivastava, A., Gutfreund, D., Ahmed, F.: Beyond statistical similarity: Rethinking metrics for deep generative models in engineering design. *Computer-Aided Design* **165**, 103609 (2023)
32. Rei, R., Stewart, C., Farinha, A.C., Lavie, A.: Comet: A neural framework for mt evaluation. *arXiv preprint arXiv:2009.09025* (2020)

33. Ren, S., Guo, D., Lu, S., Zhou, L., Liu, S., Tang, D., Sundaresan, N., Zhou, M., Blanco, A., Ma, S.: Codebleu: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297* (2020)
34. Runeson, P., Höst, M.: Guidelines for conducting and reporting case study research in software engineering. *Empirical software engineering* **14**(2), 131 (2009)
35. Sellam, T., Das, D., Parikh, A.P.: Bleurt: Learning robust metrics for text generation. *arXiv preprint arXiv:2004.04696* (2020)
36. Wang, A., Cho, K., Lewis, M.: Asking and answering questions to evaluate the factual consistency of summaries. *arXiv preprint arXiv:2004.04228* (2020)
37. Wohlin, C.: Guidelines for snowballing in systematic literature studies and a replication in software engineering. In: *Proceedings of the 18th international conference on evaluation and assessment in software engineering*, pp. 1–10 (2014)
38. Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., Regnell, B., Wesslén, A., et al.: *Experimentation in software engineering*, vol. 236. Springer (2012)
39. Xu, W., Napoles, C., Pavlick, E., Chen, Q., Callison-Burch, C.: Optimizing statistical machine translation for text simplification. *Transactions of the Association for Computational Linguistics* **4**, 401–415 (2016)
40. Yu, L., Alégroth, E., Chatzipetrou, P., Gorschek, T.: Measuring the quality of generative ai systems: Mapping metrics to quality characteristics - snowballing literature review. *Information and Software Technology* p. 107802 (2025)
41. Zhang, T., Kishore, V., Wu, F., Weinberger, K.Q., Artzi, Y.: Bertscore: Evaluating text generation with bert. *arXiv preprint arXiv:1904.09675* (2019)
42. Zhang, Z., Chen, C., Liu, B., Liao, C., Gong, Z., Yu, H., Li, J., Wang, R.: A survey on language models for code. *arXiv preprint arXiv:2311.07989* (2023)
43. Zhao, W., Peyrard, M., Liu, F., Gao, Y., Meyer, C.M., Eger, S.: Moverscore: Text generation evaluating with contextualized embeddings and earth mover distance. *arXiv preprint arXiv:1909.02622* (2019)